

Applicatielandschap

Uitleg en voorbeeld – Webshop

Hoe systemen samenwerken binnen een softwareomgeving

1. Wat is een applicatielandschap?

Een applicatielandschap is een overzicht van alle software en systemen binnen een organisatie en hoe deze met elkaar verbonden zijn. Het laat zien welke applicaties er zijn, wat ze doen en hoe ze met elkaar communiceren.

Je kunt het vergelijken met een plattegrond van een stad: elk gebouw is een systeem, en de wegen ertussen zijn de verbindingen (API's of datastromen).

Kernbegrip
Applicatielandschap = alle systemen + hun onderlinge verbindingen binnen één organisatie of product.
Doel: inzicht geven in hoe software is opgebouwd en hoe data stroomt tussen systemen.

2. De vier lagen van een applicatielandschap

Een applicatielandschap is opgebouwd uit vier lagen. Elke laag heeft een eigen verantwoordelijkheid:

Laag 1 – Frontend (presentatielaag)

Dit is het deel dat de gebruiker ziet en bedient: de website, mobiele app of het dashboard. De frontend stuurt verzoeken naar de backend en toont de ontvangen data.

- Voorbeeld: de productpagina van een webshop
- Technologieën: React, Vue, Angular, HTML/CSS

Laag 2 – Backend (logicalaag)

De backend verwerkt de logica. Hier worden regels toegepast, berekeningen gemaakt en data doorgestuurd. De frontend praat via een API met de backend.

- Voorbeeld: controleren of een product op voorraad is
- Technologieën: Node.js, Python (Django/Flask), Java (Spring)

Laag 3 – Database (datalaag)

De database slaat alle data permanent op. De backend leest en schrijft hier data naartoe via queries.

- Voorbeeld: alle bestellingen, klantgegevens en producten
- Technologieën: MySQL, PostgreSQL, MongoDB

Laag 4 – Externe services (integratielaag)

Dit zijn systemen buiten de eigen applicatie waarmee gecommuniceerd wordt, zoals een betaalservice of e-mailprovider. De communicatie verloopt via API's.

- Voorbeeld: Mollie voor betalingen, SendGrid voor e-mails
- Communicatie: REST API of webhook

3. Voorbeeld: applicatielandschap webshop

Hieronder staat een concreet voorbeeld van een webshop. De webshop bestaat uit meerdere systemen die samenwerken om een bestelling te verwerken.

3.1 Systemen in de webshop

Systeem	Functie	Laag	Technologie (voorbeeld)
Frontend	Website die de klant gebruikt	Frontend	React / Next.js
Backend API	Verwerkt logica en stuurt data door	Backend	Node.js / Express
Database	Slaat klanten, producten en orders op	Database	PostgreSQL
Betalingservice	Verwerkt de betaling van de klant	Extern	Mollie API
E-mailservice	Stuurt bevestigingsmails naar de klant	Extern	SendGrid API
Voorraadbeheer	Beheert de voorraad van producten	Backend	Intern microservice

3.2 Hoe werkt een bestelling? (stap voor stap)

Wanneer een klant een bestelling plaatst, doorloopt de data de volgende stappen:

1. De klant klikt op “Bestellen” in de Frontend.
2. De Frontend stuurt een HTTP-verzoek naar de Backend API.
3. De Backend controleert de voorraad via de Voorraadbeheer-service.
4. De Backend maakt een bestelling aan en slaat die op in de Database.
5. De Backend stuurt de klant door naar de Betalingservice (Mollie).
6. Na betaling stuurt Mollie een bevestiging terug (webhook) naar de Backend.
7. De Backend markeert de bestelling als betaald in de Database.
8. De E-mailservice stuurt een bevestigingsmail naar de klant.

3.3 Relaties en communicatie tussen systemen

Van systeem	Communicatie	Naar systeem	Wat wordt verstuurd?
Frontend	HTTP / REST API	Backend API	Bestelaanvraag, productdata opvragen
Backend API	SQL-query	Database	Bestelling opslaan, klantdata lezen
Backend API	REST API	Betalingservice	Betaallink aanmaken, status ophalen

Backend API	REST API	E-mailservice	Bevestigingsmail versturen
Backend API	Intern (RPC)	Voorraadbeheer	Voorraad controleren en bijwerken
Betalingsservice	Webhook	Backend API	Betaalstatus terugsturen

4. Wat is een API?

API staat voor Application Programming Interface. Het is een afgesproken manier waarop twee systemen met elkaar praten. Vergelijk het met een ober in een restaurant: jij (frontend) geeft je bestelling aan de ober (API), die het doorstuurt naar de keuken (backend).

Voorbeeld van een API-aanroep (REST)	
GET /api/producten	→ geeft een lijst van alle producten terug
POST /api/bestellingen	→ maakt een nieuwe bestelling aan
GET /api/bestellingen/42	→ geeft details van bestelling #42
PUT /api/bestellingen/42	→ wijzigt bestelling #42

5. Waarom is een applicatielandschap belangrijk?

- Overzicht – je ziet in één oogopslag welke systemen er zijn en wat ze doen
- Samenwerking – het laat zien hoe systemen met elkaar communiceren
- Foutopsporing – als iets mis gaat, zie je snel in welk systeem het probleem zit
- Schaalbaarheid – je kunt gericht uitbreiden zonder het hele systeem te verstoren
- Documentatie – nieuw teamleden begrijpen het systeem sneller

6. Stappen om een applicatielandschap te analyseren

9. Inventariseren – breng alle systemen in kaart (wat bestaat er al?)
10. Analyseren – beschrijf wat elk systeem doet en welke data het verwerkt
11. Relaties bepalen – hoe communiceren systemen met elkaar? Via API, database of event?
12. Visualiseren – teken een diagram (bijv. in Draw.io of Lucidchart)
13. Controleren – klopt het overzicht? Mist er een systeem of verbinding?

7. Legenda begrippen

Frontend	De gebruikersinterface; wat de gebruiker ziet en bedient
Backend	De server-kant; verwerkt logica en stuurt data aan

Database	Permanente opslag van data
API	Interface waarmee systemen met elkaar communiceren
REST API	Veelgebruikte standaard voor API's via HTTP (GET, POST, PUT, DELETE)
Webhook	Automatische melding die een systeem stuurt als er iets gebeurt
Microservice	Klein, zelfstandig systeem dat één specifieke taak uitvoert
Laag	Logische scheiding van verantwoordelijkheden in een systeem

8. Bronnen

Bass, L., Clements, P., & Kazman, R. (2012). *Software architecture in practice* (3e ed.). Addison-Wesley.

IBM. (z.d.). *Application architecture*. IBM. <https://www.ibm.com/topics/application-architecture>

Lucid Software Inc. (z.d.). *What is an application landscape?*. Lucidchart. <https://www.lucidchart.com/pages/application-landscape-diagram>

Microsoft. (z.d.). *Azure application architecture guide*. Microsoft. <https://learn.microsoft.com/en-us/azure/architecture/guide/>